

Semester Reflection Report

Semester Reflection Report: Transitioning to Modern Enterprise Architectures

Focus Area: Configuration Management, Embedded Databases, and Agile Entity Modeling

As we finalized the architecture of our car wash management system, the final major component to reflect upon was the data persistence layer. While earlier stages of the project focused on REST controllers, repositories, and frontend integration, configuring the database and modeling transactional data through the SaleRecord entity became one of the most important learning experiences of the semester. Coming from a Java EE 7 background, database integration had previously been one of the most frustrating and error-prone aspects of development. Transitioning to Spring Boot's auto-configuration features, using an embedded H2 database, and applying modern Java development practices significantly simplified the workflow and improved development efficiency. One of the biggest changes I experienced was moving away from the traditional Java EE configuration process. In earlier modules that used GlassFish or Payara servers, connecting an application to a database required multiple configuration steps. Developers needed to manually create connection pools in the application server, configure JNDI resources, and maintain large persistence.xml files for entity management. Even a small mistake in an XML configuration file or incorrect JNDI reference could prevent the application from starting correctly and often required long debugging sessions. Spring Boot introduced a completely different approach. Instead of relying on large amounts of XML configuration, the entire database connection could be established through a few simple properties inside the application.properties file: `spring.datasource.url=jdbc:h2:file:./data/queuemaster_db`
`spring.datasource.driverClassName=org.h2.Driver` `spring.datasource.username=queuemasters`
`spring.datasource.password=123` This was one of the most important lessons I learned during the semester. Spring Boot automatically reads these properties during application startup and configures essential components such as the connection pool, transaction manager, and entity manager without requiring manual setup. This "convention over configuration" philosophy allowed our team to focus more on implementing business functionality instead of spending time troubleshooting server configurations. Another important architectural decision was the use of the H2 embedded database. In previous projects, every developer needed to install and configure a separate database management system such as MySQL or Oracle before running the application locally. This often created compatibility problems because different developers could have slightly different schemas or configurations. The setup process itself also slowed down development. By using the datasource URL `jdbc:h2:file:./data/queuemaster_db`, Spring Boot automatically created a lightweight database stored directly within the project folder. Because H2 runs inside the Java Virtual Machine, the application became completely self-contained. Any team member could clone the repository, run the backend application, and immediately work with a fully functional database generated directly from the JPA entities. This experience demonstrated the importance of reducing complexity in local development environments. The embedded H2 database made development significantly faster and more convenient during the prototyping phase of the project. Although a production system would likely use a more robust database platform such as PostgreSQL or MySQL, H2 proved ideal for rapid development and testing within an academic environment. The simplicity of Spring Boot configuration was also reflected in the design of our domain models, particularly the SaleRecord entity. This entity served as the historical transaction record for the system by storing the plateNumber, washType, amount, and date for each completed car wash. This transactional information later became the foundation for generating reports and statistics within the React frontend dashboard. Compared to traditional Java development practices, the implementation of the SaleRecord entity was much cleaner and easier to maintain. Older Java applications usually required large amounts of boilerplate code, especially getter and setter methods. By using Lombok's `@Data` annotation, much of this repetitive code was generated automatically, allowing the entity class to remain concise and readable. The primary key configuration also demonstrated the advantages of modern frameworks. By applying `@Id` together with `@GeneratedValue(strategy = GenerationType.IDENTITY)`, the H2 database automatically handled ID generation for each transaction record. This removed the need for manual sequence management and ensured that every record stored in the system remained uniquely identifiable. Another important learning experience involved handling dates using the modern Java Time API. Older Java applications often relied on `java.util.Date` or `java.sql.Date`, both of which were difficult to work with and prone to timezone-related issues. These legacy date structures also required additional annotations when mapping them to database columns. Within the SaleRecord entity, we instead used `java.time.LocalDate`, which is part of the modern Java Time API introduced in Java 8. `LocalDate` is immutable, thread-safe, and far more intuitive to use than older date classes. Spring Data JPA was able to map the `LocalDate` field directly to a SQL `DATE` column without additional configuration. This highlighted how modern frameworks are designed to integrate naturally with modern Java features, resulting in cleaner and safer

code. Reflecting on the entire project, the transition from Java EE 7 to a modern React and Spring Boot architecture has been one of the most valuable learning experiences of my academic journey. At the start of the semester, I viewed frontend development mainly as a presentation layer for backend processing. Through this project, I developed an understanding of the frontend as a dynamic application capable of asynchronous communication, real-time updates, and interactive data handling. Similarly, my understanding of backend development evolved significantly. I previously viewed backend systems as configuration-heavy applications tightly coupled to large application servers. Spring Boot introduced a more lightweight and flexible approach built around auto-configured, stateless REST APIs. Working on the Live Queue tracker, Booking system, Reports dashboard, Authentication controllers, and embedded database configuration provided a complete overview of the modern software development lifecycle. The project reinforced important software engineering principles such as separation of concerns, modular design, declarative configuration, and rapid prototyping. Although adapting to these newer technologies initially required unlearning many older Java EE development habits, the experience ultimately improved both my technical skills and confidence as a developer. More importantly, it exposed me to modern industry-standard technologies and practices that are widely used in enterprise software development today. I now feel significantly more prepared to contribute to scalable and maintainable full-stack systems in a professional development environment.